

Machine Learning

End-to-End Prediction Flow & The Placement Predict Codebase Tour (CO1)



Dr. G. Omprakash

KLEF Hyderabad (Aziz Nagar) – Dept. of ECE

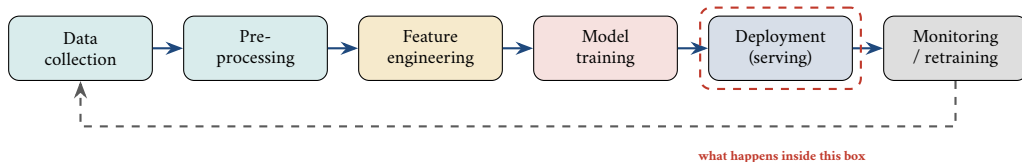
Machine Learning (25SC2107E)

- 1 Tracing a Prediction Request
- 2 Codebase Tour & Course Scope

- **Trace** a prediction request through PlacementPredict from API submission to prediction response — validation, feature transformation, model call, response.
- **Distinguish** real-time from batch prediction workflows and explain the **training-serving boundary** and the errors it causes in production.
- **Identify** the responsibilities of the data ingestion module, feature engineering pipeline, model training script, serving application and monitoring component.
- **Locate** this course in the AI landscape — where classical ML ends and Deep Learning / Generative AI / LLMs begin.

Tracing a Prediction Request

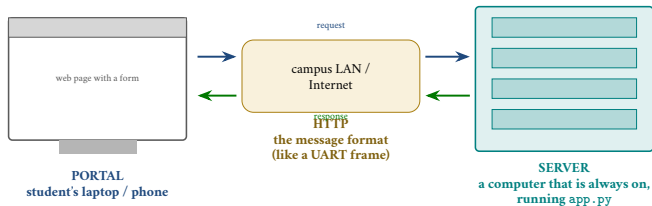
Where a Prediction Sits in the Lifecycle



Why it matters

Previous lecture asked *how is a model built?* Now the operational question: *when a real user hits “Predict”, what code runs, in what order, and where can it go wrong?*

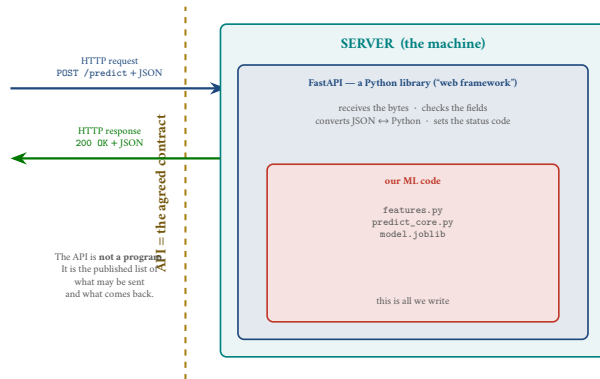
Terminology



- **Portal** — the web page the student sees: a form plus a button. It computes nothing; it only sends and displays.
- **Server** — a computer in the data centre, switched on 24×7 , waiting for messages. The trained model lives here, not on the laptop.

- **HTTP** (HyperText Transfer Protocol) — the rules for how the two machines talk: one side sends a request, the other returns a response. Nothing else is allowed.
- **JSON** (JavaScript Object Notation) — the text format the data is written in while it travels.

API and FastAPI Are Two Different Things



API — Application Programming Interface

A **published agreement** at the edge of the server: send a POST to `/predict` with these six fields, and you will get back placed and probability.

It is a specification on paper, not software — so it belongs on the **boundary**, not inside the portal or the server.

FastAPI — “Fast” + “API”

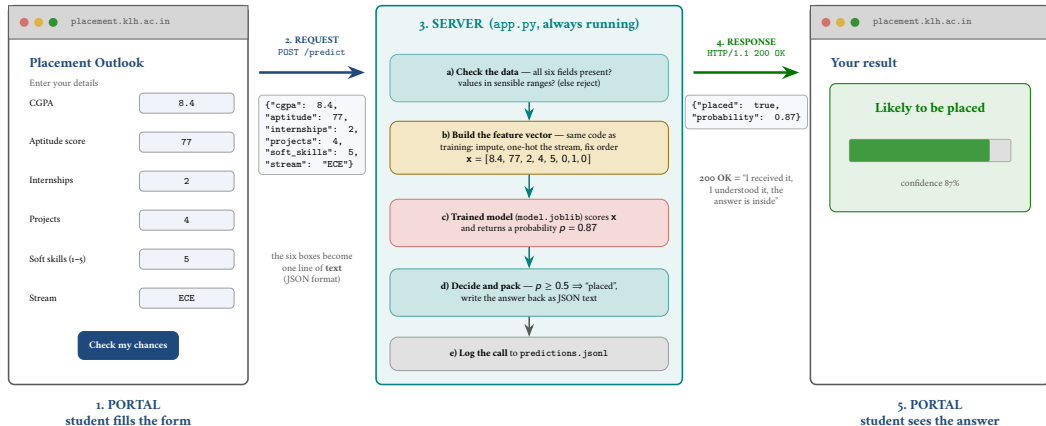
A **Python library** (not an acronym) that runs inside the server. It implements the API for us: unpacks the request, validates the six fields, calls our function, packs the reply as JSON, attaches 200 OK.

Without it we would have to write raw socket code ourselves.

One line

The **API** is the datasheet; **FastAPI** is the chip that obeys it; the **server** is the board it sits on.

The Whole Journey: Portal → Model → Portal



The whole round trip takes a few milliseconds. Steps 2 and 4 are plain text travelling over the network; steps 3(b)–3(c) are the machine learning.

What Does “200 OK” Mean?

version status code short word

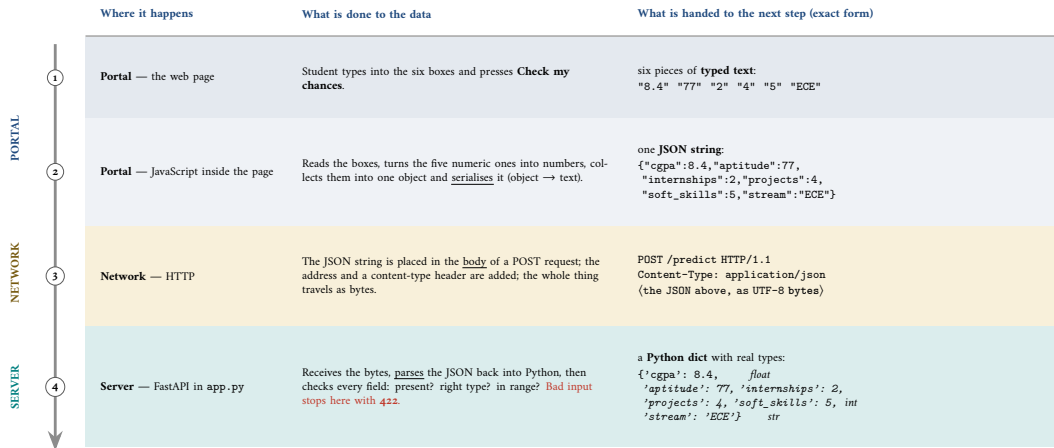


```
HTTP/1.1 200 OK
Content-Type: application/json
{"placed": true, "probability": 0.87}
```

Every HTTP reply begins with a **three-digit status code**. It tells the portal what kind of reply this is, before the portal even looks at the contents.

Code	Family	Meaning here
200	success	prediction is inside, use it
422	your fault	a field is missing or invalid
404	your fault	wrong address (typo in /predict)
500	our fault	the server code crashed

How the Data Changes Shape — Part 1: Portal → Server



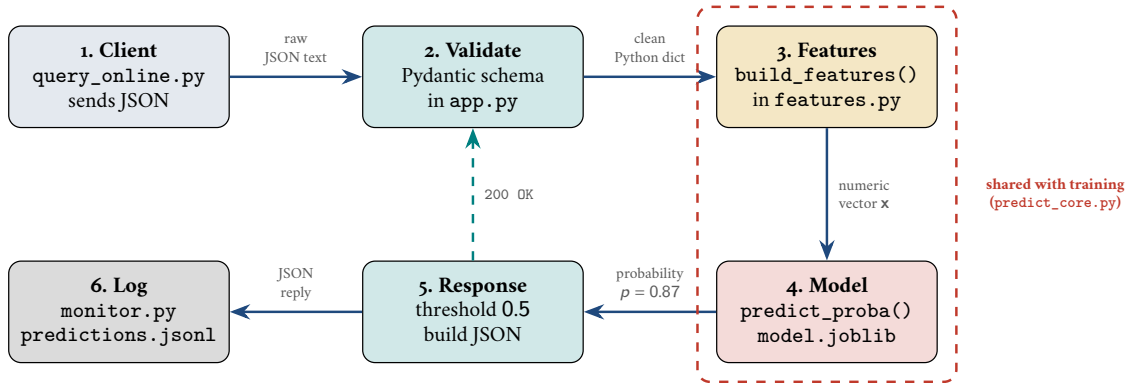
So far nothing has been computed. Steps 1–4 only change the container: typed text → JSON text → bytes → Python dict.

How the Data Changes Shape — Part 2: Server → Portal

	Where it happens	What is done to the data	What is handed to the next step (exact form)
SERVER	5 Server — <code>features.py</code> , <code>build_features()</code>	Fill a missing CGPA with the <u>training</u> mean; one-hot encode <u>stream</u> into three o/1 columns; place all columns in exactly the order used during training.	a NumPy array , shape (1,8): [[8.4, 77., 2., 4., 5., 0., 1., 0.]] <i>cgpa apt int proj soft CSE ECE MECH</i>
	6 Server — the model, <code>model.joblib</code>	The trained estimator scores that single row with <code>predict_proba(X)</code> . This is the <u>only</u> step that uses anything learned from data.	[[0.13, 0.87]] → keep column 1 → p = 0.87 a single float
	7 Server — <code>predict_core.py</code> + FastAPI	Apply the 0.5 threshold, build a small dict, <u>serialise</u> it back to JSON, attach the status code, and log one line to <code>predictions.jsonl</code> .	HTTP/1.1 200 OK Content-Type: application/json {“placed”: true, “probability”: 0.87}
PORTAL	8 Portal — JavaScript again	Parses the JSON reply and writes the two values into the page. No ML runs on the student’s laptop at any point.	pixels on the screen: “Likely to be placed”, confidence bar filled to 87%

Full chain: typed text → JSON text → bytes → dict → array → float → JSON text → pixels. The model sees only step 5.

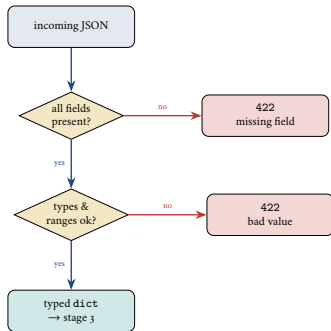
Trace of a Prediction Request: API → Response



Read the diagram

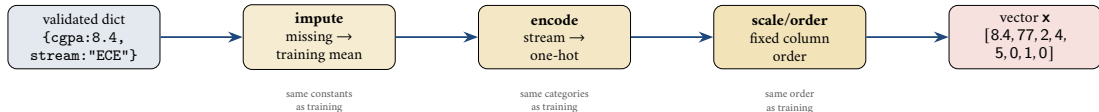
Stages 1–2 are web engineering. Stages 3–4 are machine learning. Stages 5–6 are operations. A production ML engineer owns all six.

Stage 2 in Detail: Input Validation



- FastAPI uses a **Pydantic model** to declare the expected schema (`cgpa: float, stream: str, ...`).
- Validation happens before any ML code runs — a bad request never reaches the model.
- Rejecting early gives a clear 422 error instead of a meaningless prediction from garbage input.

Stage 3 in Detail: Transforming Inputs into Features

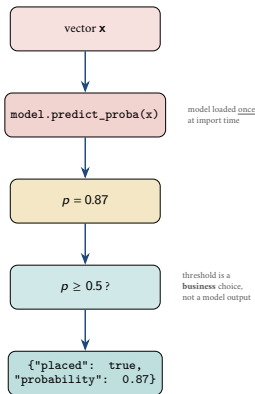


- The model does not understand "ECE" — only numbers.
- Column order matters: the model has no names, only positions.

The golden rule

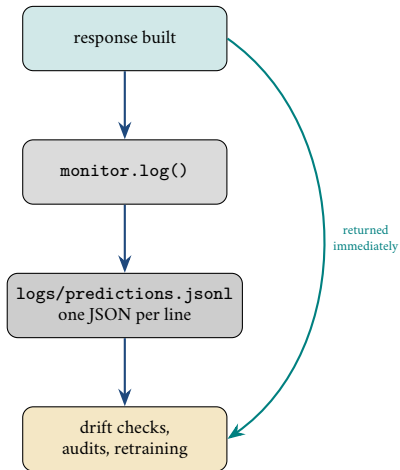
Anything learned from the training data (means, category lists, scaling constants) must be **saved with the model** and reused at serving time.

Stage 4-5: Model Call and Response Construction



- `joblib.load()` is slow — do it at startup, never per request.
- The model returns a **probability**; the decision comes from a threshold you choose.
- Returning the probability (not just the label) lets the portal show “strong / borderline / weak” instead of a bare yes/no.

Stage 6: Logging — and Why It Is Not Optional



- Log the **inputs**, the **probability** and the **model version**.
- Without logs you cannot answer: “why did the system reject this student in March?”
- Logs are the raw material for detecting **data drift** (Session 5+).

Why it matters

Logging must not block the response. Write the line after replying, or asynchronously.

Real-Time vs Batch Prediction

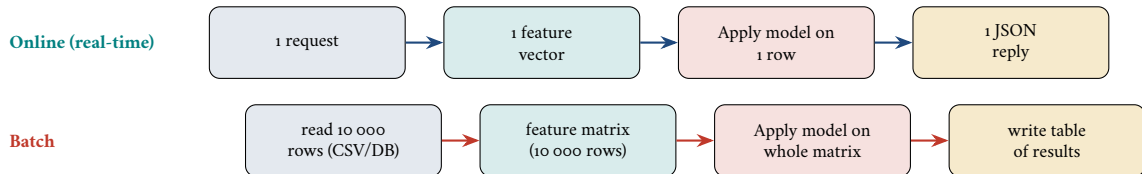


Figure: Online vs. batch. Same four ideas; the difference is one row, answered now versus many rows, written to a file.

Real-time (online)

- prediction on demand, per request
- latency: milliseconds
- e.g. a student checks their placement outlook in the portal
- needs: fast features, always-on service

Batch (offline)

- predictions computed for all rows on a schedule
- latency: minutes–hours, throughput matters
- e.g. nightly placement-risk report for all 5,000 final-year students
- needs: cheap compute, simple retries

The Training–Serving Boundary

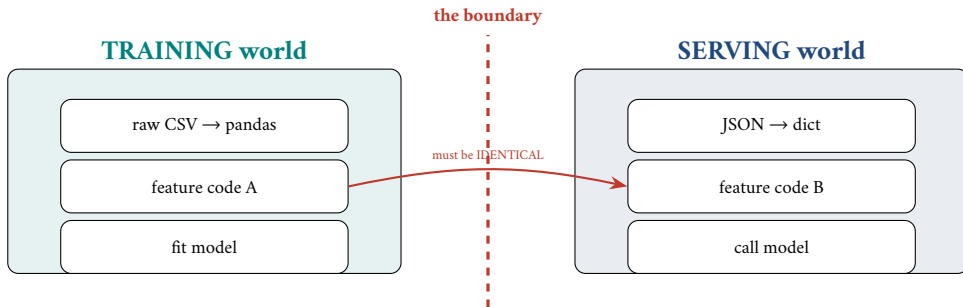


Figure: If “feature code A” (training) and “feature code B” (serving) are not exactly the same, you get **training–serving skew**.

- **Training world** — offline, big historical dataset, `train.py`, pandas over a whole CSV.
- **Serving world** — online, one student at a time, inside the FastAPI app, under time pressure.

Missing Values

When a student's CGPA is missing, what do we fill it with, and what happens if training and serving fill it differently?

Filling missing values

- **During training:** standard procedure is to fill a missing CGPA with the *training mean*
- **During serving** we must do the same
- If serving instead fills with zero, we get the `zero_impute skew`
 - The predictions for missing-CGPA students collapse.

Fork and clone the repository https://github.com/omprakash65/PlacementPredict_SkewLab.git

Zero Impute Skew

apt	intern	proj	soft	stream	$P(\text{mean})$	$P(\text{zero})$	result
77	2	4	5	ECE	0.995	0.806	same
54	1	2	2	MECH	0.626	0.032	FLIPPED → not placed
52	1	6	2	CSE	0.925	0.195	FLIPPED → not placed
47	1	3	1	MECH	0.511	0.020	FLIPPED → not placed
63	1	1	4	MECH	0.858	0.105	FLIPPED → not placed
86	3	2	5	ECE	0.996	0.840	same

Table: The 0.5 decision line splits placed / not placed. Filling with 0 drags strong candidates below it. In total, **28 of the 45** missing-CGPA students flip to “not placed” under the bug.

Codebase Tour & Course Scope

The PlacementPredict Repository Layout

PlacementPredict/

```

make_data.py    create dummy data
ingest.py       read the CSV

features.py     build feature vector

train.py        fit + save model
predict_core.py shared scoring code
  
```

```

app.py          FastAPI server
query_online.py HTTP client
predict_offline.py offline scorer
batch_predict.py offline batch job
  
```

```

monitor.py      log predictions
requirements.txt libraries needed
  
```

data/

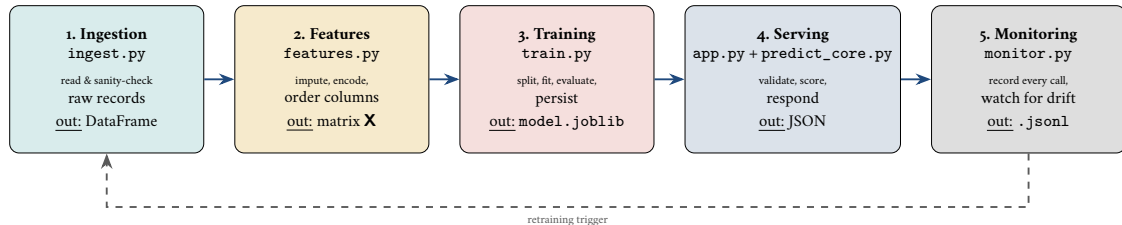
```

    students.csv    historical (with label)
    batch_input.csv new students (no label)
models/            model.joblib (made by train.py)
logs/              predictions.jsonl (made at run time)
  
```

Link: <https://github.com/omprakash65/Machine-Learning/tree/main/PlacementPredict>

The PlacementPredict repository. Colours match the roles: **data in**, **features**, **model**, **serving**, monitoring. You can run any of them with `python <file>.py`.

Five Components and Their Responsibilities

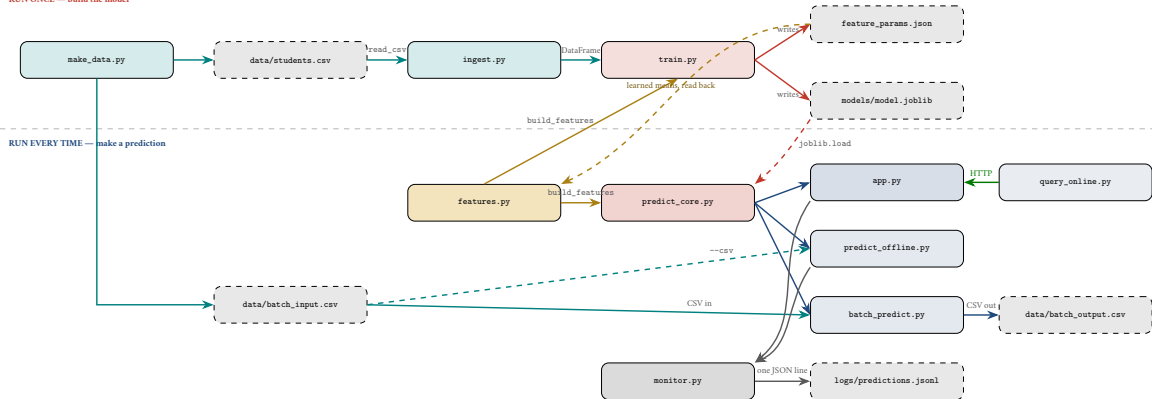


The contract view

Each component is defined by **what it consumes and what it produces**. If you can state those two things, you understand the module — regardless of the code inside.

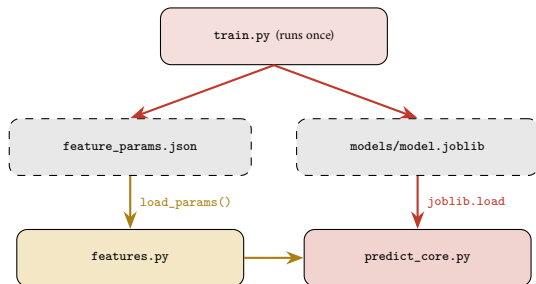
How the Files Actually Connect

RUN ONCE — build the model



Dashed boxes are **artifacts** created at run time; solid boxes are **code**. `batch_predict.py` does not log; `query_online.py` never imports `predict_core` — it only speaks HTTP to `app.py`.

The Two Artifacts That Cross the Boundary



Both are written by `train.py` and read back at serving time. Delete either one and the server cannot answer.

`feature_params.json`

The constants learned from the training set:

```
{"cgpa_mean": 7.070, "aptitude_mean": 63.99}
```

`train.py` computes them from `students.csv` and writes the file; `features.py` reads it once at import and uses the same numbers to fill missing values at serving time.

`models/model.joblib`

The fitted `LogisticRegression` object. `predict_core.py` loads it once at import, never per request.

The point

A trained model is **not** one file. It is the estimator plus every constant the feature code needs. Ship both, or you ship skew.

Task

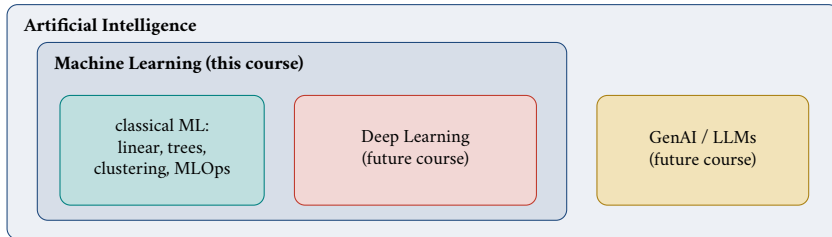
Fork the folder and clone it to your PC:
<https://github.com/omprakash65/PlacementPredict.git>

- Setup (one time)
 - pip install -r requirements.txt
- Prepare data and train the model (one time)
 - python make_data.py
 - python train.py
- Make predictions — OFFLINE
 - Single Prediction
 - python predict_offline.py
 - Batch Prediction
 - python batch_predict.py
- Make predictions — ONLINE
 - Terminal 1 — Start the Server
 - uvicorn app:app -reload
 - Open <http://127.0.0.1:8000/docs> in a browser for a clickable test page.
 - Terminal 2 — send queries
 - python query_online.py -batch
 - python query_online.py -csv data/batch_input.csv

Nomenclature

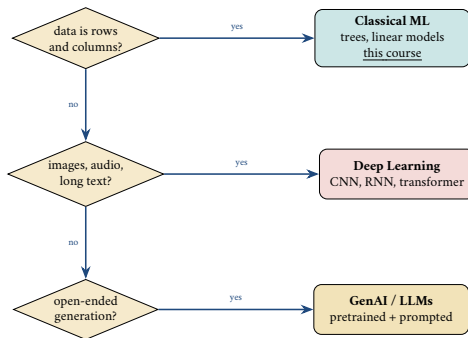
- **Seed** is a starting number that makes “random” results repeatable.
 - Computers generate random numbers from a formula that begins at the seed.
 - If everyone uses the same seed, everyone gets the same “random” dummy data.
 - We used 42 in `make_data.py`
- **JSON** (JavaScript Object Notation) is a text format for organizing data using key: value pairs
 - Different programs can exchange information. It looks like a Python dictionary: `{"cgpa": 8.4, "internships": 2}`.
- **joblib** is a Python library that saves a Python object to a file on disk and loads it back later.
 - In the project, `train.py` learns the model, then does `joblib.dump(model, "model.joblib")` to freeze that trained model into a file.
 - Later, `app.py` does `joblib.load(...)` to get it back instantly.
- **Endpoint** is a named URL a server listens on, e.g. `POST /predict`.
- **Latency** is the time between sending a request and receiving the reply.
- **jsonl** means “JSON Lines” — a file where each line is one complete JSON object.
 - `monitor.py` writes one line per prediction to `predictions.jsonl`. Handy for logs: keep appending new lines without rewriting the file, and read one record at a time.

Scope of this Course & the AI Landscape



- **In scope:** tabular supervised + unsupervised learning, evaluation, deployment, monitoring.
- **Out of scope** (but previewed): neural networks, embeddings, transformers, LLMs.
- The engineering skills (pipelines, serving, drift) transfer unchanged to DL/GenAI systems.

Where Classical ML Ends



Key Insight

On tabular data of the size a college actually has, gradient-boosted trees usually **beat** deep networks — and are far cheaper to run and explain. Choosing the simpler method is a professional skill, not a limitation.

ML vs Deep Learning vs Generative AI

	Classical ML	Deep Learning	GenAI / LLMs
Typical data	tables, few thousand rows	images, audio, text, millions of samples	internet-scale text / code
Features	hand-engineered (<code>features.py</code>)	learned by the network	learned; the prompt is the interface
Output	label or number	label, box, embedding	new text, image, code
Compute	laptop	GPU cluster	pretrained by others
Explainability	high	low	very low

The constant

The six-stage request trace, the training–serving boundary and the monitoring loop are identical in all three columns. That is why we teach them first.

Thankyou